KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY, KUMASI.

COLLEGE OF ENGINEERING

DEPARTMENT OF ELECTRICAL/ELECTRONIC ENGINEERING

Project Report

COE 381 — Microprocessors

Group 20

Date: 29/03/26

NAME	INDEX
Mondjro Christian	7329723
Osei Afrifa Andy	7333223
Angel Aseda Pinto	7334523
Tetekpor Petrova Dzidedi	7336223
Nyumuyo Osborne	7331123
Norbi Helina Asantewaa	7330523
Opoku Randy Agyemang	7332823
Alhassan Mohammed	7329623
Owusu Abena Achiaa Clenda	7333723

Contents

1 Introduction **2**

2 Problem Statement **2**

3 Methodology **2**
- 3.1 System Design Overview 2
- 3.2 Hardware Connections 3
- 3.3 Software Libraries 3
- 3.4 MQ-2 Sensor Calibration 4
- 3.5 Control Logic 4
- 3.6 Full Annotated Source Code 4

4 Circuit Schematic Description **7**

5 Reference Links **7**

6 Results and Testing **8**

7 Conclusion **8**
- 7.1 Summary of Findings 8
- 7.2 Challenges Encountered 8
- 7.3 Possible Improvements 8

1. Introduction

Gas leaks and smoke present serious hazards in domestic, industrial, and public environments. Early detection is critical in preventing injury, property damage, and loss of life. This project develops a low-cost, microcontroller-based gas-detection and alarm system that continuously monitors airborne gas concentrations, alerts occupants through an audible buzzer and an LED, and displays a human-readable status message on a liquid-crystal display (LCD). Smoke was selected as the primary test medium owing to its practical relevance and the ready response of the chosen sensor to combustion products.

2. Problem Statement

Conventional gas detectors available commercially are often expensive, offer limited feedback to the user, and provide no straightforward interface for adjusting sensitivity thresholds. In many resource-constrained settings, organisations and households require an affordable solution that:

- Continuously monitors gas concentrations in real time.
- Distinguishes between *safe*, *warning*, and *danger* concentration levels and communicates these states clearly.
- Produces an audible and visual alarm when unsafe levels are detected.
- Allows occupants to *acknowledge and mute* the alarm manually without removing power from the device.
- Can be assembled from widely available, low-cost components.

3. Methodology

3.1 System Design Overview

The system is built around an **Arduino Uno** microcontroller. The principal components are:

- **MQ-2 Gas Sensor** — detects combustible gases and smoke by measuring changes in electrical resistance.
- **16×2 I²C LCD Module** — displays the current status string (`SAFE`, `WARNING`, or `DANGER!!!`).
- **Buzzer** — generates an audible alarm on GPIO pin 3.
- **LED** — provides a visual alarm signal (driven alongside the buzzer on pin 13, read back on the same pin as `buzzpinread`).
- **Push Button** — allows the operator to mute an active alarm.
- **9V Battery** — powers the arduino uno.

Table 1: Pin Assignments on the Arduino Uno

Component	**Arduino Pin**	**Mode**
MQ-2 Analog Output	A0	Analog Input
Buzzer and LED	3	Digital Output
Buzzer and LED	13	Digital Input
Push Button	11	Digital Input/Output
LCD (SDA)	A4	I^2C Data
LCD (SCL)	A5	I^2C Clock

3.2 Hardware Connections

The push button is connected between pin 11 and ground; enabled via `digitalWrite(pushbutton, HIGH)`, so the button reads `LOW` when pressed.

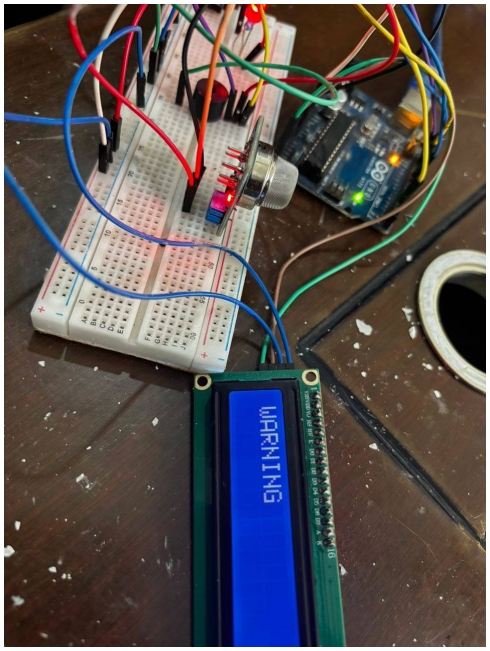

Figure 1: Picture of Connections

3.3 Software Libraries

Two external libraries were used:

1. **MQ2 Arduino Library** by *labay10* — provides a high-level interface for reading the MQ-2 sensor and includes built-in calibration logic.
   https://github.com/labay11/MQ-2-sensor-library
2. **LiquidCrystal_I2C** by *Frank de Brabander / marcoschwartz* — simplifies communication with I^2C LCD modules.
   https://github.com/marcoschwartz/LiquidCrystal_I2C

3.4 MQ-2 Sensor Calibration

The MQ-2 sensor outputs an analog voltage proportional to gas concentration. The `MQ2` library converts this into a concentration value in **parts per million (ppm)** through an internal calibration routine invoked by `mq2.begin()`. This routine samples the sensor in clean air to establish a baseline resistance R_0, and then applies the following relationship derived from the sensor's datasheet:

$$\text{ppm} = a \cdot \left(\frac{R_S}{R_0}\right)^b$$

where R_S is the instantaneous sensor resistance and a, b are gas-specific constants included in the library.

The threshold values used in this project were set empirically during testing with smoke:

- `threshold1 = 1 ppm` — transition from `SAFE` to `WARNING`.
- `threshold2 = 4 ppm` — transition from `WARNING` to `DANGER!!!`.

These values may be adjusted in the source code to suit different environments or stricter safety requirements.

3.5 Control Logic

1. The sensor is read each cycle via `mq2.readSmoke()`.
2. If the concentration is below `threshold1` *and* the alarm has not been muted, the LCD displays `SAFE` and the buzzer is OFF.
3. If the concentration is between the two thresholds, the buzzer turns ON and the LCD shows `WARNING`.
4. If the concentration exceeds `threshold2`, the buzzer remains ON and the LCD shows `DANGER!!!`.
5. Pressing the pushbutton while the buzzer is active sets `alarmmute = 1`, silencing the alarm and clearing the display.
6. Pressing the button again while the buzzer is inactive resets `alarmmute = 0`, restoring normal operation.

3.6 Full Annotated Source Code

```
#include "MQ2.h"                // MQ-2 sensor functions
#include <LiquidCrystal_I2C.h> // I2C LCD functions

/* -- Pin assignments
   ------------------------------------------------------ */
int buzzerpin     = 3;    // Buzzer output pin
int pushbutton    = 11;   // Push-button input pin
int gasSensorpin  = A0;   // MQ-2 analog input
int buzzpinread   = 13;   // Read-back pin for alarm network

```

```
/* -- State variables ------------------------------------------------------ */
int alarmmute = 0;        // 0 = alarm active, 1 = alarm muted
int delaytime = 1000;     // Main loop delay (ms)
int pushdetect;           // Stores button read
int readbuzzpin;          // Stores alarm network read

/* -- Gas concentration thresholds (ppm) ---------------------------------- */
float threshold1 = 1;     // SAFE -> WARNING boundary
float threshold2 = 4;     // WARNING -> DANGER boundary

/* -- Object initialisation ----------------------------------------------- */
LiquidCrystal_I2C lcd(0x27, 16, 2); // I2C address 0x27, 16 cols, 2 rows
MQ2 mq2(gasSensorpin);

void setup() {
  Serial.begin(9600);
  pinMode(pushbutton,  INPUT);
  pinMode(buzzerpin,     OUTPUT);
  pinMode(buzzpinread, INPUT);
  digitalWrite(pushbutton, HIGH);

  lcd.init();        // Initialise LCD
  lcd.backlight();
  lcd.clear();

  mq2.begin();       // Calibrate and initialise MQ-2
}

void loop() {
  lcd.setCursor(0, 1);
  float smoke = mq2.readSmoke(); // Read smoke concentration (ppm)

  /* -- Tiered alert logic ------------------------------------------------ */
  if (smoke < threshold1 && alarmmute == 0) {
    lcd.clear();
    lcd.print("SAFE");
    digitalWrite(buzzerpin, LOW);

  } else if (smoke >= threshold1 && smoke < threshold2 && alarmmute == 0) {
    lcd.clear();
    digitalWrite(buzzerpin, HIGH);
    lcd.print("WARNING");

  } else if (smoke >= threshold2 && alarmmute == 0) {
```

```
    lcd.clear();
    digitalWrite(buzzerpin, HIGH);
    lcd.print("DANGER!!!");
  }

  /* -- digital Reads from push button and alarm network
     ------------------------------------------------- */
  pushdetect  = digitalRead(pushbutton);
  readbuzzpin = digitalRead(buzzpinread);

  /* -- Alarm mute / reset logic
     ---------------------------------------- */
  if (pushdetect == 0 && readbuzzpin == 1) {
    // Button pressed while alarm active -> mute
    alarmmute = 1;
    digitalWrite(buzzerpin, LOW);
    lcd.clear();
    delay(delaytime);

  } else if (pushdetect == 0 && readbuzzpin == 0) {
    // Button pressed while alarm inactive -> un-mute
    alarmmute = 0;
  }

  delay(delaytime);
}
```

Listing 1: Complete Arduino sketch for the gas detection system

4. Circuit Schematic Description

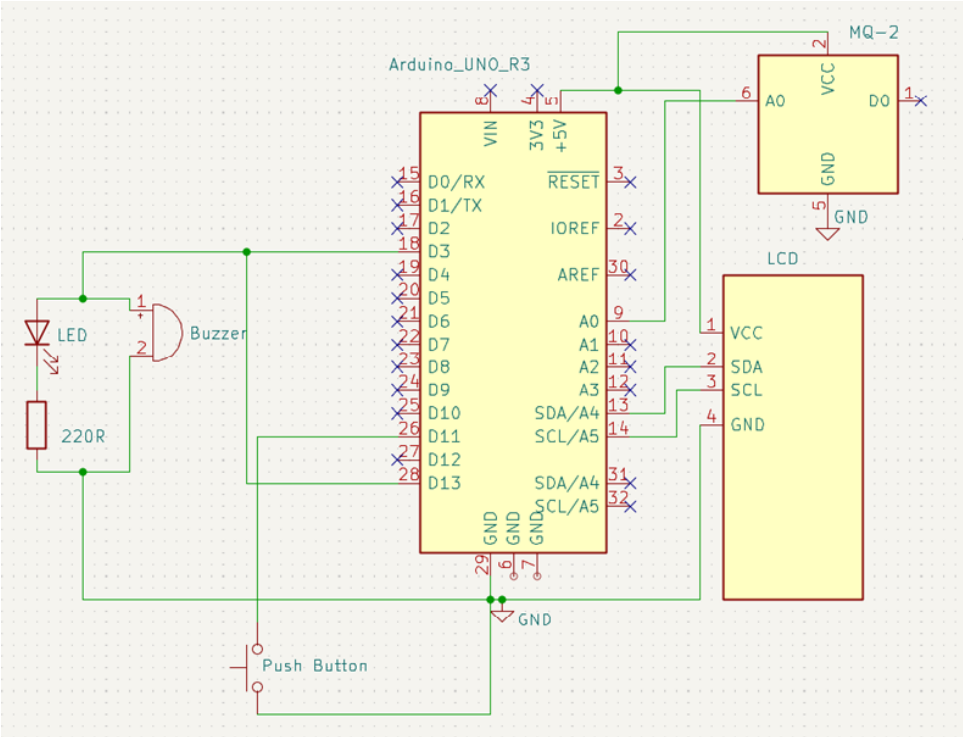

Figure 2: Circuit Schematic

- The **MQ-2 sensor module** (4-pin breakout: VCC, GND, DO, AO) connects VCC to the Arduino 5 V rail, GND to common ground, and the AO (analog output) to pin A0. The DO (digital output) pin is not used.
- The **buzzer** connects its positive terminal to pin 13 and pin 3 while its negative terminal is connected ground.
- The **LED** connects its anode through a 220 Ω resistor to pin 13 and pin 3 while its cathode is connected ground.
- The **push button** connects one terminal to pin 11 and the other to ground. The internal pull-up means no external resistor is required.
- The **I²C LCD module** connects VCC to 5 V, GND to common ground, SDA to Arduino A4, and SCL to Arduino A5.

5. Reference Links

1. **MQ-2 Library (labay10)** — source of the `MQ2.h` library used in this project, including calibration constants and `readSmoke()` implementation.
   https://github.com/labay11/MQ-2-sensor-library
2. **MQ-2 Datasheet** — sensor characteristics and recommendations.
   http://sandboxelectronics.com/?p=165

3. **LiquidCrystal_I2C Library** — I^2C LCD driver for Arduino.
   https://github.com/marcoschwartz/LiquidCrystal_I2C
4. **Tinkercad Circuits** — online tool used to prototype and simulate the schematic before physical assembly.
   https://www.tinkercad.com/circuits

6. Results and Testing

During testing, a small amount of smoke (produced by burning a piece of paper) was used to verify system behaviour. The observed results were as follows:

- In clean air the LCD correctly displayed `SAFE` and the buzzer remained silent.
- Introducing light smoke caused the reading to cross `threshold1`, triggering the `WARNING` message and activating the buzzer.
- Holding the smoke source closer caused the reading to exceed `threshold2`, escalating the display to `DANGER!!!`.
- Pressing the push button while the alarm was active successfully muted the buzzer and cleared the screen.
- Pressing the button a second time in the absence of an active alarm re-enabled normal monitoring.

The system responded within one loop cycle (approximately 1 s) to changes in concentration, which is adequate for a low-cost safety demonstration.

7. Conclusion

7.1 Summary of Findings

This project successfully demonstrated a functional, tiered gas-detection and alarm system. The MQ-2 sensor provided reliable smoke detection, and the I^2C LCD offered clear status feedback without consuming many digital I/O pins. The mute function improved usability by allowing occupants to acknowledge an alarm without turning the device off.

7.2 Challenges Encountered

- **Threshold calibration.** Selecting appropriate ppm thresholds required iterative testing(a lot of burning to generate smoke) because the raw ppm values reported by the library are sensitive to temperature, humidity, and the specific batch of sensor used.

7.3 Possible Improvements

- **SMS notification.** Integrating an ESP8266 module would allow the system to send remote alerts when gas is detected.
- **Data logging.** Storing timestamped readings on an SD card or to a cloud platform would enable trend analysis and post-incident investigation.

- **Adjustable thresholds via keypad.** Allowing end users to set thresholds without modifying the source code would improve deployability.
- **Enclosure.** Mounting the components in a ventilated housing would make the device suitable for real-world deployment.